

# Лабораторная работа №4

## Криптосистема Мак-Элиса на квазициклических MDPC-кодах (QC-MDPC)

*Пояснительная записка: как и почему это работает. Руководство к самостоятельной реализации*

---

### Содержание

|                                                  |    |
|--------------------------------------------------|----|
| 1. О чём документ .....                          | 2  |
| 2. Обозначения .....                             | 2  |
| 3. Параметры .....                               | 3  |
| 3.1. Один алгоритм – разные параметры .....      | 3  |
| 4. Кольцо $R = \text{GF}(2)[x]/(x^r - 1)$ .....  | 4  |
| 4.1. Элементы и операции .....                   | 4  |
| 4.2. Циркулянтные матрицы .....                  | 4  |
| 4.3. Выбор $r$ и обратимость .....               | 4  |
| 4.4. Квадрат и обращение .....                   | 5  |
| 5. Код QC-MDPC .....                             | 5  |
| 5.1. Определение .....                           | 5  |
| 5.2. Почему «умеренная плотность» .....          | 5  |
| 6. Декодер: переворот битов .....                | 6  |
| 6.1. Синдром .....                               | 6  |
| 6.2. Счётчик и почему он работает .....          | 6  |
| 6.3. Алгоритм .....                              | 6  |
| 6.4. Выбор порога .....                          | 7  |
| 6.5. Отказы декодирования .....                  | 7  |
| 7. Криптосистема .....                           | 7  |
| 7.1. Проблема прямолинейного варианта .....      | 7  |
| 7.2. Генерация ключей .....                      | 8  |
| 7.3. Зашифрование .....                          | 8  |
| 7.4. Расшифрование и его корректность .....      | 8  |
| 7.5. Вариант ВКЕ (для справки) .....             | 8  |
| 8. Кодирование сообщений .....                   | 8  |
| 9. Стойкость .....                               | 9  |
| 9.1. Две задачи противника .....                 | 9  |
| 9.2. Учебные параметры небезопасны .....         | 9  |
| 9.3. Атака реакцией .....                        | 9  |
| 9.4. Слабости учебной схемы .....                | 9  |
| 10. План реализации .....                        | 9  |
| 10.1. Модули .....                               | 9  |
| 10.2. Порядок работы и тесты .....               | 9  |
| 10.3. Подводные камни .....                      | 10 |
| 11. Мини-пример ( $r = 11, d = 3, t = 1$ ) ..... | 10 |
| 12. Вопросы на защите .....                      | 11 |

## 1. О чём документ

Криптосистема Мак-Элиса шифрует так: сообщение кодируется помехоустойчивым кодом, к результату добавляются  $t$  случайных ошибок. Исправить их может только тот, кто знает скрытую структуру кода. В оригинале (1978) используются коды Гоппы; здесь – **квазициклические коды с проверочной матрицей умеренной плотности** (QC-MDPC, Мисочки, Тийиш, Сендрие, Баррето, 2013). На них построен BIKE – финалист четвёртого раунда конкурса NIST по постквантовой криптографии.

|                                       | Коды Гоппы                                 | QC-MDPC                                                |
|---------------------------------------|--------------------------------------------|--------------------------------------------------------|
| Арифметика                            | поле $\text{GF}(2^m)$ и многочлены над ним | двоичные многочлены по модулю $x^r - 1$ : сдвиги и XOR |
| Декодер                               | алгоритм Паттерсона (алгебраический)       | переворот битов (итеративный, вероятностный)           |
| Соккрытие структуры                   | матрицы $S$ и $P$ , метод Гаусса           | секрет – сама разреженность; нужна только маска $a$    |
| Открытый ключ при 128 битах стойкости | 261 120 байт                               | около 3 КБ (в форме BIKE – 1,5 КБ)                     |
| Слабое место                          | размер ключа                               | декодер изредка ошибается                              |

Документ описывает математику, алгоритмы по шагам и объясняет, почему каждый шаг работает. Кода в нём нет: реализация – ваша. План и подводные камни – в разделе 10. Все числа проверены моделированием.

## 2. Обозначения

| Символ             | Смысл                                                                           |
|--------------------|---------------------------------------------------------------------------------|
| $r$                | длина блока: простое число, для которого 2 – первообразный корень по модулю $r$ |
| $R$                | кольцо $\text{GF}(2)[x]/(x^r - 1)$ : двоичные многочлены степени меньше $r$     |
| $n = 2r$           | длина кода                                                                      |
| $h_0, h_1$         | секретные разреженные многочлены, в каждом ровно $d$ единиц; $d$ нечётно        |
| $w = 2d$           | вес строки проверочной матрицы                                                  |
| $t$                | число ошибок, добавляемых при шифровании                                        |
| $q = h_0 h_1^{-1}$ | многочлен, задающий открытый код                                                |
| $a$                | случайный обратимый плотный многочлен – маска сообщения                         |
| $(A, B) = (a, aq)$ | открытый ключ                                                                   |
| $m$                | блок сообщения: $r$ бит, то есть элемент $R$                                    |
| $e_0, e_1$         | векторы ошибок, суммарно ровно $t$ единиц                                       |
| $ v $              | вес: число единиц                                                               |
| $\text{supp}(v)$   | носитель: множество позиций единиц                                              |

### Соглашения

- Многочлен  $a = a_0 + a_1x + \dots + a_{r-1}x^{r-1}$  и битовый вектор  $(a_0, \dots, a_{r-1})$  – одно и то же. Бит  $i$  – коэффициент при  $x^i$ .
- Все коэффициенты из  $\text{GF}(2)$ : сложение и вычитание – XOR,  $-1 = 1$ .

- Показатели степеней всегда берутся по модулю  $r$ :  $x^r = 1$ .

### 3. Параметры

| Набор           | $r$    | $d$ | $w$ | $t$ | Назначение                                                       |
|-----------------|--------|-----|-----|-----|------------------------------------------------------------------|
| мини            | 11     | 3   | 6   | 1   | ручной разбор (раздел 11)                                        |
| учебный         | 587    | 15  | 30  | 16  | отладка: всё видно, декодируется за миллисекунды                 |
| средний         | 1019   | 21  | 42  | 26  | проверка на больших блоках                                       |
| BIKE, уровень 1 | 12 323 | 71  | 142 | 134 | около 128 бит стойкости; открытый ключ – $2r$ бит, примерно 3 КБ |

Все значения  $r$  в таблице – простые числа, для которых 2 является первообразным корнем (почему это важно – раздел 4.3). Подходящие простые в диапазоне 100–700: 101, 107, 131, 139, 149, 163, 173, 179, 181, 197, 211, 227, 269, 293, 317, 347, 349, 373, 379, 389, 419, 421, 443, 461, 467, 491, 509, 523, 541, 547, 557, 563, 587, 613, 619, 653, 659, 661, 677, 701.

#### 3.1. Один алгоритм – разные параметры

Кольцо, обращение, синдром, счётчики, шифрование и расшифрование одинаковы для любого набора: правильная реализация не должна зависеть от  $r$ ,  $d$  и  $t$ . Но параметры – не три независимые ручки, и три вещи с ними не переносятся.

1. **Порог декодера.** Пороговая функция BIKE (раздел 6.4) подобрана только под  $r = 12323$ ,  $d = 71$ . На других наборах она не работает вовсе: например, при  $d = 15$  счётчик не превышает 15, а порог не опускается ниже 36, так что ни один бит не перевернётся. Порог привязан к набору параметров. Вариант «переворачивать максимум» работает везде без настройки, но на уровне 1 требует около 64 итераций вместо 3.
2. **Число ошибок  $t$ .** Декодер справляется, только пока ошибок мало относительно  $r$  и  $d$ . При  $r = 587$ ,  $d = 15$ :  $t = 16$  – отказов нет,  $t = 24$  – редкие отказы,  $t = 30$  – отказ в 62 % случаев (раздел 6.5).  $t$  выбирается вместе с  $r$  и  $d$ , а не отдельно.
3. **Условия на  $r$  и  $d$ .**  $r$  простое и 2 – первообразный корень по модулю  $r$ ;  $d$  нечётное. Нарушение ломает обращение (раздел 4.3) – это не ошибка реализации, а неподходящие параметры.

**Зачем нужны маленькие наборы, если алгоритм тот же.** Не ради скорости: на Rust уровень 1 работает за миллисекунды на блок.

- **Проверка вручную.** При  $r = 11$  вектор уместается в строку, и мини-пример (раздел 11) сверяется до бита. На 12 323 битах ошибку глазами не найти.
- **Полный перебор в тестах.** При  $r = 587$  можно проверить одиночную ошибку во всех  $2r = 1174$  позициях и прогнать тысячи блоков за секунды. Так ловится, например, несогласованное направление сдвига.
- **Границы машинных слов.** При  $r = 11 < 64$  вектор помещается в одно слово, и перенос битов между словами при сдвиге вообще не проверяется. При  $r = 587$  – 10 слов по 64 бита, сдвиги пересекают границы, а последнее слово неполное. Мини-примера поэтому недостаточно, нужен и учебный набор.
- **Демонстрация отказов.** На маленьких параметрах легко зависить  $t$  и показать, что программа честно сообщает об отказе. На уровне 1 отказ случайно не увидеть.

#### Итог

Отлаживать – на  $r = 11 \rightarrow 587 \rightarrow 1019$ , демонстрировать и сдавать – на уровне 1. Только он крипто-стойкий: учебные наборы вскрываются ISD (раздел 9.2).

## 4. Кольцо $R = \text{GF}(2)[x]/(x^r - 1)$

### 4.1. Элементы и операции

Элемент  $R$  – многочлен степени меньше  $r$  с битовыми коэффициентами, то есть вектор из  $r$  бит. Операции:

- **Сложение** – XOR векторов.
- **Умножение на  $x^j$**  – циклический сдвиг на  $j$  позиций в сторону старших индексов:  $x^j \cdot x^i = x^{(i+j) \bmod r}$ , и бит, «вылезший» за позицию  $r - 1$ , возвращается в начало, потому что  $x^r = 1$ .
- **Произведение** – циклическая свёртка:

$$(ab)_k = \sum_{i+j \equiv k \pmod{r}} a_i b_j \pmod{2}.$$

Раскрытие скобок даёт практичное правило: **произведение – это XOR сдвигов одного множителя на позиции единиц другого**,

$$a \cdot b = \sum_{i \in \text{supp}(a)} x^i \cdot b.$$

Отсюда стоимость: если  $a$  разрежен ( $d$  единиц), то  $ab$  – это  $d$  сдвигов и XOR. Для  $d = 71$ ,  $r = 12323$  – мгновенно. Произведение двух плотных многочленов – около  $r/2$  сдвигов, что тоже быстро при хранении вектора в машинных словах.

Кольцо **коммутативно**:  $ab = ba$ . На этом держится вся схема (раздел 5).

### 4.2. Циркулянтные матрицы

Многочлену  $a$  соответствует циркулянтная матрица  $r \times r$ , у которой строка  $i$  – это вектор  $x^i a$ , то есть  $a$ , сдвинутый на  $i$ :

$$\text{circ}(a) = \begin{pmatrix} a_0 & a_1 & \dots & a_{r-1} \\ a_{r-1} & a_0 & \dots & a_{r-2} \\ \vdots & & \ddots & \vdots \\ a_1 & a_2 & \dots & a_0 \end{pmatrix}.$$

Умножение вектора  $u$  на такую матрицу – это произведение многочленов  $ua$ , а произведение циркулянтов – циркулянт произведения. Матрица из нескольких циркулянтных блоков называется **квазициклической** – отсюда «QC» в названии. Практический смысл: матрица  $r \times r$  хранится одной строкой из  $r$  бит, а не  $r^2$  битами. Именно поэтому ключ QC-MDPC в сто раз меньше ключа Гоппы.

### 4.3. Выбор $r$ и обратимость

Для реализации нужно уметь обращать элементы  $R$ . Кольцо  $R$  не поле: например,  $(1+x)(1+x+\dots+x^{r-1}) = 1+x^r = 0$  – произведение ненулевых элементов даёт ноль. Какие элементы обратимы, определяется разложением  $x^r - 1$  на неприводимые множители.

Всегда  $x^r - 1 = (x+1)\Phi_r(x)$ , где  $\Phi_r(x) = 1+x+\dots+x^{r-1}$ . Если  $r$  простое и  $2$  – **первообразный корень по модулю  $r$**  (то есть  $2^k \not\equiv 1$  при  $0 < k < r-1$ ), то  $\Phi_r$  неприводим над  $\text{GF}(2)$ . Тогда по китайской теореме об остатках

$$R \cong \text{GF}(2)[x]/(x+1) \times \text{GF}(2)[x]/(\Phi_r) \cong \text{GF}(2) \times \text{GF}(2^{r-1}).$$

Элемент обратим тогда и только тогда, когда обе его компоненты ненулевые:

- компонента в  $\text{GF}(2)$  – это  $a(1) = a_0 + a_1 + \dots + a_{r-1}$ , то есть **чётность веса**. Она ненулевая ровно при нечётном  $|a|$ ;
- компонента в  $\text{GF}(2^{r-1})$  – это  $a \bmod \Phi_r$ . Она нулевая лишь при  $a = 0$  и  $a = \Phi_r$  (все единицы).

#### Правило обратимости

При правильно выбранном  $r$  многочлен  $a \in R$  обратим  $\Leftrightarrow$  его вес нечётен и  $a$  не равен вектору из одних единиц. Поэтому  $d$  выбирается нечётным, а случайную маску  $a$  генерируют до получения нечётного веса.

**Зачем так строго выбирать  $r$ .** Если у  $x^r - 1$  много мелких неприводимых множителей, то в  $R$  много необратимых элементов, и появляются атаки, «сворачивающие» задачу по малому множителю. С двумя множителями кольцо устроено максимально просто и против таких атак защищено.

#### 4.4. Квадрат и обращение

**Квадрат – перестановка битов.** В характеристике 2 перекрёстные члены исчезают:

$$\left( \sum_i a_i x^i \right)^2 = \sum_i a_i x^{2i \bmod r}.$$

Возвести в квадрат – значит переставить бит  $i$  в позицию  $2i \bmod r$ . Вес при этом не меняется.

**Обращение: способ 1 – расширенный алгоритм Евклида** над  $\text{GF}(2)[x]$  для пары  $(x^r - 1, a)$ . Поддерживаются остатки  $r_i$  и коэффициенты  $v_i$  с инвариантом

$$r_i \equiv v_i \cdot a \pmod{x^r - 1},$$

начиная с  $r_{-1} = x^r - 1$ ,  $v_{-1} = 0$  и  $r_0 = a$ ,  $v_0 = 1$ . На каждом шаге  $r_{i-1}$  делится на  $r_i$  с остатком:  $r_{i+1} = r_{i-1} - q_i r_i$ ,  $v_{i+1} = v_{i-1} - q_i v_i$ . Когда остаток становится равен 1, соответствующий  $v_i$  и есть  $a^{-1}$ . Если остаток обнулится раньше, чем стал единицей, значит  $\gcd(a, x^r - 1) \neq 1$  и  $a$  необратим. Над  $\text{GF}(2)$  деление в столбик особенно просто: старший коэффициент всегда 1, и каждый шаг деления – это XOR сдвинутого делителя.

Тонкость:  $x^r - 1$  имеет степень ровно  $r$  и не помещается в  $r$  бит, поэтому для Евклида удобно хранить многочлены «с запасом» в один бит или обрабатывать первый шаг отдельно.

**Способ 2 – возведение в степень.** Для обратимого  $a$  в обеих компонентах выполнено  $a^{2^{r-1}-1} = 1$ , откуда

$$a^{-1} = a^{2^{r-1}-2}.$$

Возведение в квадрат дёшево (перестановка), но прямой метод «квадрат и умножение» требует около  $r$  плотных умножений. Цепочка Итоха – Цудзии сокращает их до  $O(\log r)$ . Для первой реализации Евклид понятнее; второй способ удобен как независимая проверка в тестах.

## 5. Код QC-MDPC

### 5.1. Определение

Секрет – два разреженных многочлена  $h_0, h_1 \in R$  веса  $d$ . Код длины  $n = 2r$  состоит из пар  $(u, v) \in R^2$ , для которых

$$u h_0 + v h_1 = 0.$$

В матричной записи это проверочная матрица из двух циркулянтных блоков,  $H = [\text{circ}(h_0) \mid \text{circ}(h_1)]$ , размера  $r \times 2r$ . В каждой её строке  $w = 2d$  единиц, отсюда и вес строки в обозначениях.

**Размерность и порождающая матрица.** Если  $h_1$  обратим (нечётный  $d$ ), то для любого  $u$  существует ровно один подходящий  $v = u h_0 h_1^{-1}$ . Значит, код имеет размерность  $r$ , и каждое кодовое слово – это

$$(u, uq), \quad q = h_0 h_1^{-1}.$$

Проверка:  $u h_0 + u q h_1 = u h_0 + u h_0 h_1^{-1} h_1 = u h_0 + u h_0 = 0$ . Здесь использованы коммутативность и  $z + z = 0$ .

Многочлен  $q$  плотный: обращение «размазывает» разреженный  $h_1$  по всем позициям (в разделе 11 при  $r = 11$  вес  $q$  равен 5 из 11). По  $q$  код можно построить, но быстрый декодер – только по  $h_0, h_1$ .

### 5.2. Почему «умеренная плотность»

Декодеру (раздел 6) нужны разреженные проверки: чем меньше  $w$ , тем надёжнее каждый счётчик. Противнику же разреженность помогает найти ключ, поскольку  $(h_1, h_0)$  – кодовое слово **открытого** кода:

$$(h_1, h_1 q) = (h_1, h_1 h_0 h_1^{-1}) = (h_1, h_0).$$

Его вес всего  $w$ , тогда как типичное кодовое слово имеет вес около  $r$ . Восстановление ключа – это поиск слова малого веса в коде по  $q$ .

- LDPC (слабая плотность,  $w$  порядка 10): такие слова находятся быстро – схема взломана (2000).
- Плотная проверочная матрица: декодер перестаёт работать.
- MDPC:  $w \approx \sqrt{n}$ . Для  $r = 12323$  имеем  $\sqrt{24646} \approx 157$ , в BIKE  $w = 142$ . Слово такого веса перебором уже не найти, а декодер ещё работает.

## 6. Декодер: переворот битов

### 6.1. Синдром

Пусть получено  $(c_0, c_1) = (u, uq) + (e_0, e_1)$ . Синдром

$$s = c_0 h_0 + c_1 h_1 = e_0 h_0 + e_1 h_1$$

зависит только от ошибок: вклад кодового слова обнуляется по определению кода.

**Какие проверки затрагивает бит.** Ошибка в бите  $j$  части  $e_0$  – это слагаемое  $x^j$ , и её вклад в синдром –  $x^j h_0$ , то есть  $h_0$ , сдвинутый на  $j$ . Бит  $j$  участвует в проверках (позициях синдрома)

$$P_0(j) = \{(j+p) \bmod r : p \in \text{supp}(h_0)\},$$

а бит  $j$  части  $e_1$  – в  $P_1(j) = \{(j+p) \bmod r : p \in \text{supp}(h_1)\}$ . Каждый бит участвует ровно в  $d$  проверках, каждая проверка содержит  $w$  бит. Бит синдрома равен 1 («проверка не выполнена»), когда в ней нечётное число ошибок.

### 6.2. Счётчик и почему он работает

**Счётчик** бита – число невыполненных проверок среди его  $d$  проверок:

$$\text{urs}_0(j) = |\text{supp}(s) \cap P_0(j)|, \quad \text{urs}_1(j) = |\text{supp}(s) \cap P_1(j)|.$$

**Интуиция.** Если бит ошибочен, то каждая его проверка «заражена» им самим и не выполнена, если остальные ошибки попали в неё чётное число раз. Если бит верный, проверка не выполнена лишь при нечётном числе чужих ошибок. Ошибки редки, поэтому в проверку в среднем попадает  $\lambda \approx tw/n$  ошибок, и в модели Пуассона

$$P(\text{не выполнена} \mid \text{бит ошибочен}) \approx \frac{1 + e^{-2\lambda}}{2}, \quad P(\text{не выполнена} \mid \text{бит верен}) \approx \frac{1 - e^{-2\lambda}}{2}.$$

Для BIKE уровня 1  $\lambda \approx 134 \cdot 142 / 24646 \approx 0.77$ , вероятности – 0,61 и 0,39. Счётчики – суммы  $d = 71$  таких событий, и моделирование подтверждает теорию:

| Бит       | Средний счётчик | Стандартное отклонение |
|-----------|-----------------|------------------------|
| ошибочный | 43,2            | 4,1                    |
| верный    | 28,0            | 4,1                    |

Распределения разделены примерно на 3,7 стандартного отклонения. Порог между ними отделяет большинство ошибочных битов от верных. Каждый верный переворот уменьшает число ошибок,  $\lambda$  падает, распределение растёт – и за несколько итераций ошибки исчезают.

### 6.3. Алгоритм

1. Вычислить синдром  $s = c_0 h_0 + c_1 h_1$ . Обнулить найденные ошибки  $\hat{e}_0 = \hat{e}_1 = 0$ .
2. Если  $s = 0$  – закончить (шаг 7).
3. Для всех  $2r$  битов вычислить счётчики  $\text{urs}_0(j)$  и  $\text{urs}_1(j)$ .
4. Выбрать порог  $T$  (раздел 6.4).
5. Каждый бит со счётчиком не меньше  $T$  перевернуть в  $\hat{e}$  и обновить синдром: для бита  $j$  части 0 прибавить к  $s$  многочлен  $x^j h_0$ , для части 1 –  $x^j h_1$ . Все счётчики этой итерации вычисляются до переворотов.
6. Если число итераций не превысило предел, вернуться к шагу 2; иначе – отказ декодирования.
7. Проверить  $|\hat{e}_0| + |\hat{e}_1| = t$ . Если нет – отказ.

Шаг 5 можно сделать и проще: перевернуть биты в  $\hat{e}$  и пересчитать  $s$  заново по формуле  $s = (c_0 + \hat{e}_0)h_0 + (c_1 + \hat{e}_1)h_1$ . Это медленнее, но исключает ошибки в инкрементальном обновлении.

## 6.4. Выбор порога

**Вариант «переворачивать максимум»** (для учебных параметров):  $T = \max_j \max(\text{urs}_0(j), \text{urs}_1(j))$ . Переворачиваются только биты с наибольшим счётчиком. Это надёжно и без настройки, но переворачивается мало битов за итерацию, поэтому итераций порядка  $t/2$ .

**Пороговая функция ВКЕ** (для  $r = 12323$ ): порог зависит от веса синдрома,

$$T = \max(\lfloor 0.0069722 \cdot |s| + 13.530 \rfloor, 36).$$

Константы взяты из спецификации ВКЕ для уровня 1; сверьтесь с её актуальной версией. В начале  $|s| \approx 4870$  и  $T = 47$ : переворачивается около 22 % ошибочных битов и практически ни одного верного. Дальше синдром легчает, порог опускается к 36, и декодирование заканчивается за 3–4 итерации.

### Функция ВКЕ -- только для уровня 1

Счётчик бита не может превышать  $d$ , а функция ВКЕ никогда не даёт порог ниже 36. Для любого набора с  $d < 36$ , то есть для всех учебных, декодер с этой функцией не перевернёт ни одного бита и не сработает ни разу. Константы подобраны под  $r = 12323$ ,  $d = 71$  и на другие параметры не масштабируются. Практичное решение – реализовать оба варианта порога и хранить выбор вместе с набором параметров: «максимум» для учебных наборов, функция ВКЕ для уровня 1.

Результаты моделирования (случайные ключи и ошибки):

| $r, d, t$       | Порог       | Отказов  | Итераций в среднем | Максимум |
|-----------------|-------------|----------|--------------------|----------|
| 587, 15, 16     | максимум    | 0 из 300 | 8,3                | 12       |
| 1019, 21, 26    | максимум    | 0 из 300 | 13,6               | 20       |
| 12 323, 71, 134 | функция ВКЕ | 0 из 200 | 3,0                | 4        |
| 12 323, 71, 134 | максимум    | 0 из 10  | 63,5               | 78       |

## 6.5. Отказы декодирования

Декодер вероятностный: при неудачном расположении ошибок он может не сойтись или сойтись к неверному  $\hat{e}$ . Вероятность этого (DFR, decoding failure rate) резко растёт, если  $t$  завышено:

| $r, d$   | $t$ | Отказов из 300 |
|----------|-----|----------------|
| 587, 15  | 16  | 0              |
| 587, 15  | 24  | 1              |
| 587, 15  | 30  | 186            |
| 1019, 21 | 26  | 0              |
| 1019, 21 | 36  | 19             |
| 1019, 21 | 44  | 286            |

Для ВКЕ улучшенный декодер (Black-Gray-Flip) доводит DFR до  $2^{-128}$ ; простой вариант выше заметно хуже, но при уровне 1 на сотнях испытаний отказов не даёт. Отказ в программе – это сообщение «блок не расшифровался», а не неверный текст: проверка шага 7 плюс контроль  $c_1 = mAq$  (раздел 7.4) ловят ложные решения.

**Сложность** одной итерации –  $2r$  счётчиков по  $d$  слагаемых, то есть  $2rd \approx 1.75 \cdot 10^6$  операций для уровня 1.

## 7. Криптосистема

### 7.1. Проблема прямолинейного варианта

Самый прямой вариант – шифровать кодовым словом  $(m, mq)$ :  $c_0 = m + e_0$ ,  $c_1 = mq + e_1$ . Он **небезопасен**:  $c_0$  – это само сообщение, в котором перевёрнуто около  $t/2 \approx 67$  бит из 12 323. Больше 99 % открытого

текста читается напрямую. Это следствие систематического вида порождающей матрицы, то же самое, что было бы с кодом Гоппы без матрицы  $S$ .

Лекарство – маскирующий обратимый многочлен  $a$ , аналог матрицы  $S$ : вместо  $m$  кодируется  $ma$ . Плотное  $a$  «размазывает» сообщение, и в  $c_0$  его уже не видно. Перестановка  $P$  из схемы с Гоппой здесь не нужна: структура кода и так скрыта, поскольку по  $q$  разреженные  $h_0, h_1$  не найти (раздел 5.2).

## 7.2. Генерация ключей

1. Выбрать случайные  $h_0, h_1$  ровно с  $d$  единицами каждый ( $d$  нечётно).
2. Вычислить  $h_1^{-1}$  (раздел 4.4) и  $q = h_0 h_1^{-1}$ .
3. Выбрать случайный плотный  $a$  нечётного веса, не равный вектору из одних единиц, и вычислить  $a^{-1}$ .
4. Открытый ключ:  $(A, B) = (a, aq)$  и число  $t$ . Закрытый ключ:  $(h_0, h_1, a^{-1})$ .

Проверки для тестов:  $h_1 h_1^{-1} = 1$ ,  $aa^{-1} = 1$ ,  $h_1 q = h_0$ .

## 7.3. Зашифрование

Блок  $m$  – вектор из  $r$  бит.

1. Выбрать  $t$  различных случайных позиций из  $2r$ ; позиции  $< r$  образуют  $e_0$ , остальные (минус  $r$ ) –  $e_1$ .
2.  $c_0 = mA + e_0$ ,  $c_1 = mB + e_1$ .

Шифроблок – пара  $(c_0, c_1)$ ,  $2r$  бит.

## 7.4. Расшифрование и его корректность

1. Синдром  $s = c_0 h_0 + c_1 h_1$ .
2. Декодер находит  $(e_0, e_1)$  или сообщает об отказе.
3.  $m = (c_0 + e_0) \cdot a^{-1}$ .
4. Контроль (рекомендуется):  $(c_1 + e_1)$  должно равняться  $mB$ .

**Почему синдром зависит только от ошибок:**

$$s = (ma + e_0)h_0 + (maq + e_1)h_1 = mah_0 + mah_0 h_1^{-1} h_1 + e_0 h_0 + e_1 h_1 = e_0 h_0 + e_1 h_1.$$

Два слагаемых  $mah_0$  взаимно уничтожаются. Декодер восстанавливает  $e_0$ , откуда  $c_0 + e_0 = ma$  и  $m = (ma)a^{-1}$ .

### Маска открыта -- и это нормально

Многочлен  $a$  входит в открытый ключ, так что  $a^{-1}$  может вычислить любой. Но это ничего не даёт:  $c_0 a^{-1} = m + e_0 a^{-1}$ , а  $e_0 a^{-1}$  – плотный шум. Снять его можно только зная  $e_0$ , то есть решив задачу декодирования. Маска нужна не для секретности ключа, а чтобы открытый текст не проступал в шифротексте.

## 7.5. Вариант VIKЕ (для справки)

В реальном VIKЕ сообщения нет вообще. Отправитель выбирает случайные  $(e_0, e_1)$  и посылает  $c = e_0 + e_1 h$ , где  $h = h_1 h_0^{-1}$ . Общий ключ – хэш от  $(e_0, e_1)$ , а сами данные шифруются симметричным шифром (в работе №2 есть BelT). Получатель вычисляет  $ch_0 = e_0 h_0 + e_1 h_1$  – тот же синдром – и декодирует. Это схема инкапсуляции ключа (KEM): шифротекст всего  $r$  бит, а одноразовые ключи закрывают атаку реакцией (раздел 9.3).

## 8. Кодирование сообщений

- **Порядок битов.** Бит  $8k + b$  потока – бит  $b$  байта  $k$  (младший первым).
- **Дополнение.** К битам сообщения дописывается единица и нули до длины, кратной  $r$ . Дополнение есть всегда, поэтому при расшифровании последняя единица однозначно отмечает конец данных.
- **Блоки.** Каждый блок из  $r$  бит шифруется в  $2r$  бит, которые записываются в  $\lceil 2r/8 \rceil$  байт или как две половины по  $\lceil r/8 \rceil$  байт. Неиспользуемые биты последнего байта должны быть нулевыми.
- **Расширение.** Шифротекст ровно вдвое длиннее дополненного текста. Для  $r = 12323$  блок вмещает 1540 байт текста.

## 9. Стойкость

### 9.1. Две задачи противника

1. **Восстановление ключа:** найти слово веса  $w$  в коде  $\{(u, uq)\}$ . Таких слов не одно, а  $r$  штук:  $(x^j h_1, x^j h_0)$  для всех сдвигов. Это квазициклическая структура, и она даёт противнику выигрыш примерно в  $r$  раз.
2. **Расшифрование без ключа:** найти  $(e_0, e_1)$  веса  $t$  по синдрому относительно открытого кода. Циклическость снова помогает, примерно в  $\sqrt{r}$  раз.

Лучшие известные алгоритмы для обеих задач – декодирование по информационным множествам (ISD). Параметры BIKE уровня 1 подобраны с учётом этих выигрышей под 128 бит стойкости; публичных взломов нет.

### 9.2. Учебные параметры небезопасны

При  $r = 587$  слово веса  $w = 30$  или ошибка веса 16 в коде длины 1174 находятся ISD за разумное время. Учебные наборы нужны только для отладки и наглядности – как пресет `toy` в работе с Гоппой.

### 9.3. Атака реакцией

Вероятность отказа декодера зависит от **расстояний** между единицами в  $h_0$  и  $h_1$ . Если противник многократно отправляет шифротексты с подобранными ошибками и видит, какие из них не расшифровались, он восстанавливает эти расстояния, а по ним – ключ (Гиней, Джонсон, Штанкевич, 2016). Защита: одноразовые ключи (как в BIKE) или декодер с пренебрежимо малым DFR.

### 9.4. Слабости учебной схемы

- **Податливость:**  $(c_0 + m'A, c_1 + m'B)$  – корректный шифротекст сообщения  $m + m'$ .
- **Проверка догадки:** зная кандидата  $m$ , противник проверяет, что  $(c_0 + mA, c_1 + mB)$  имеет вес ровно  $t$ .
- **Повторная отправка:** два шифрования одного  $m$  дают  $(c_0 + c_0', c_1 + c_1') = (e_0 + e_0', e_1 + e_1')$  – позиции ошибок раскрываются.

Всё это свойства схемы Мак-Элиса «как в учебнике» с любым кодом. На практике их закрывают схемой KEM (раздел 7.5) или конверсиями вроде Кобары – Имаи.

## 10. План реализации

### 10.1. Модули

1. **Многочлены в  $R$ .** Хранение: массив машинных слов, бит  $i$  – в слове  $i/64$ , разряд  $i \bmod 64$ ; биты за позицией  $r - 1$  всегда нулевые. Операции: XOR, вес, чтение и установка бита, циклический сдвиг, умножение (через сдвиги по единицам одного из множителей), квадрат, обращение, список позиций единиц.
2. **Генерация:** случайный вектор ровно с  $k$  единицами (выбор  $k$  различных позиций), случайный плотный вектор нечётного веса.
3. **Декодер:** синдром, счётчики, порог, итерации, проверка веса.
4. **Криптосистема:** ключи, шифрование и расшифрование блока.
5. **Сообщения:** байты в биты, дополнение, блоки, упаковка шифроблоков.

### 10.2. Порядок работы и тесты

1. **Кольцо.** Тесты:  $x^j \cdot x^k = x^{(j+k) \bmod r}$ ; сдвиг на  $r$  – тождество;  $ab = ba$ ;  $a(b + c) = ab + ac$ ; квадрат совпадает с  $a \cdot a$ ;  $aa^{-1} = 1$  для случайных  $a$  нечётного веса; вектор чётного веса необратим;  $a^{-1}$  по Евклиду совпадает с  $a^{2^{r-1}-2}$  при малом  $r$ .
2. **Мини-пример** из раздела 11 – все промежуточные значения должны совпасть. Он помещается в одно машинное слово, поэтому тесты кольца обязательно повторить при  $r = 587$ : там сдвиги пересекают границы слов.
3. **Декодер** на учебных параметрах: одиночная ошибка находится в каждой из  $2r$  позиций (полный перебор); при  $t = 16$ ,  $r = 587$  сотни случайных испытаний без отказов.
4. **Криптосистема:** зашифрование и расшифрование случайных блоков;  $h_1 q = h_0$ ; синдром шифротекста равен  $e_0 h_0 + e_1 h_1$ .

5. **Сообщения:** пустое, одна буква, длина, кратная блоку, кириллица, файл.
6. **Уровень 1:** несколько сотен блоков без отказов; замер времени.
7. **Демонстрация отказов:** завышенное  $t$  на учебных параметрах даёт отказы, и программа сообщает о них, а не выдаёт мусор.

### 10.3. Подводные камни

- **Направление сдвига.** Умножение на  $x^j$  и множества  $P(j)$  в счётчиках должны использовать **одно и то же** соглашение. Самая частая ошибка: синдром считается сдвигом «вверх», а счётчик – «вниз». Тогда декодер не сходится даже на одной ошибке. Тест «одна ошибка в каждой позиции» ловит это сразу.
- **Хвост последнего слова.** После сдвига биты за позицией  $r - 1$  нужно переносить в начало и обнулять, иначе вес и сравнение векторов врут.
- **Чётность весов.**  $d$  нечётно, маска  $a$  нечётного веса, иначе обращения нет.
- **Счётчики до переворотов.** В одной итерации все счётчики считаются по одному синдрому, и только потом биты переворачиваются.
- **Предел итераций.** Без него декодер при отказе зависнет.
- **Ровно  $t$  ошибок.** Позиции выбираются без повторов, иначе вес меньше  $t$ .
- **Степень  $x^r - 1$ .** В Евклиде этот многочлен имеет  $r + 1$  коэффициент.
- **Порог привязан к параметрам.** Функция BIKE на учебных наборах не переворачивает ни одного бита (раздел 6.4). Если декодер «ничего не делает» на маленьком  $r$ , первым делом проверьте порог.
- **$t$  согласовано с  $r$  и  $d$ .** Отказы при своих параметрах – не обязательно ошибка в коде: сначала сравните  $t$  с таблицей раздела 6.5.

## 11. Мини-пример ( $r = 11, d = 3, t = 1$ )

2 – первообразный корень по модулю 11, поэтому  $x^{11} - 1 = (x + 1)(1 + x + \dots + x^{10})$  и правило обратимости работает.

**Ключ.**

$$h_0 = 1 + x + x^2, \quad h_1 = 1 + x + x^3, \quad h_1^{-1} = 1 + x + x^3 + x^6 + x^7 + x^8 + x^{10}.$$

Проверка  $h_1 h_1^{-1} = 1$  – хороший первый тест для обращения. Многочлен  $1 + x$  (чётный вес) необратим:  $(1 + x) \cdot \Phi_{11} = 0$ .

$$q = h_0 h_1^{-1} = x + x^4 + x^5 + x^6 + x^8, \quad h_1 q = 1 + x + x^2 = h_0.$$

Разреженные  $h_0, h_1$  (по 3 единицы) дали плотный  $q$  (5 из 11) – структура кода не видна. Квадрат для проверки:  $h_1^2 = 1 + x^2 + x^6$ , то есть показатели  $\{0, 1, 3\}$  удвоились до  $\{0, 2, 6\}$ .

**Кодирование и ошибка.** Здесь для наглядности без маски ( $a = 1$ ). Сообщение  $m = 1 + x^2 + x^3 + x^8$ ,  $m q = 1 + x^2 + x^8 + x^{10}$ , ошибка  $e_1 = x^5$ ,  $e_0 = 0$ :

$$c_0 = 1 + x^2 + x^3 + x^8, \quad c_1 = 1 + x^2 + x^5 + x^8 + x^{10}.$$

**Синдром.**  $s = c_0 h_0 + c_1 h_1 = x^5 + x^6 + x^8$  – ровно  $x^5 h_1$ , то есть  $h_1 = 1 + x + x^3$ , сдвинутый на 5.

**Счётчики.**

| $j$               | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-------------------|---|---|---|---|---|---|---|---|---|---|----|
| $\text{wpc}_0(j)$ | 0 | 0 | 0 | 1 | 2 | 2 | 2 | 1 | 1 | 0 | 0  |
| $\text{wpc}_1(j)$ | 0 | 0 | 1 | 1 | 1 | 3 | 1 | 1 | 1 | 0 | 0  |

Максимум 3 – у бита 5 части 1: все три его проверки не выполнены. У остальных битов не больше двух невыполненных проверок, потому что их проверки пересекаются с проверками ошибочного бита лишь частично. Переворачиваем бит, синдром обнуляется,  $e_1 = x^5$  найден, и  $m = c_0 + e_0 = c_0$ .

## 12. Вопросы на защите

### ? Почему QC-MDPC проще кодов Гоппы?

Вся арифметика – двоичные многочлены по модулю  $x^r - 1$ : сдвиги и XOR. Нет поля  $\text{GF}(2^m)$ , многочленов над ним и алгебраического декодера Паттерсона. Декодер – подсчёт невыполненных проверок и переворот битов.

### ? Что значит «квазициклический» и что это даёт?

Проверочная матрица составлена из циркулянтных блоков: каждая строка блока – циклический сдвиг предыдущей. Такой блок задаётся одной строкой, то есть многочленом, а умножение на него – произведение многочленов. Ключ получается в  $r$  раз компактнее: около 3 КБ вместо сотен килобайт.

### ? Что значит «умеренная плотность» и почему не меньше и не больше?

Вес строки проверочной матрицы  $w \approx \sqrt{n}$ . При меньшем весе (LDPC) секретный  $(h_1, h_0)$  – слишком лёгкое кодовое слово открытого кода, и его находят. При большем весе счётчики ошибочных и верных битов перестают различаться, и декодер не работает.

### ? Почему $r$ – простое, а 2 – первообразный корень по модулю $r$ ?

Тогда  $x^r - 1 = (x + 1)\Phi_r$  с неприводимым  $\Phi_r$ , и  $R \cong \text{GF}(2) \times \text{GF}(2^{r-1})$ . Обратимость проверяется одной чётностью веса, а у кольца нет мелких подструктур, через которые можно было бы упростить атаку.

### ? Работает ли реализация одинаково для любых $r, d, t$ ?

Алгоритм один и тот же, но параметры согласованы между собой.  $r$  простое с первообразным корнем 2,  $d$  нечётное;  $t$  достаточно мало относительно  $r$  и  $d$ , иначе декодер отказывает. Порог декодера привязан к набору: функция BIKE подобрана под  $d = 71$  и при  $d < 36$  не переворачивает ни одного бита. Маленькие наборы нужны для отладки – ручной сверки, полного перебора ошибок и проверки сдвигов через границы машинных слов, а продемонстрировать стойкость – только на уровне 1.

### ? Почему $h_1$ и маска $a$ должны иметь нечётный вес?

Многочлен чётного веса обращается в ноль при  $x = 1$ , то есть делится на  $x + 1$ , а значит, необратим. Нужны  $h_1^{-1}$  (для  $q$ ) и  $a^{-1}$  (для расшифрования).

### ? Как вычисляется обратный элемент?

Расширенным алгоритмом Евклида для  $(x^r - 1, a)$  с инвариантом  $r_i \equiv v_i a$ : когда остаток становится 1,  $v_i = a^{-1}$ . Альтернатива –  $a^{-1} = a^{2^{r-1}-2}$ , потому что порядок обратимого элемента делит  $2^{r-1} - 1$ .

### ? Почему синдром не зависит от сообщения?

$c_0 h_0 + c_1 h_1 = m a h_0 + m a q h_1 + e_0 h_0 + e_1 h_1$ , а  $m a q h_1 = m a h_0 h_1^{-1} h_1 = m a h_0$ . Два одинаковых слагаемых в характеристике 2 дают ноль. Остаётся  $e_0 h_0 + e_1 h_1$ .

### ? Как работает декодер переворота битов?

Для каждого бита считается, сколько из его  $d$  проверок не выполнены. Ошибочный бит «портит» все свои проверки, и его счётчик в среднем выше (43 против 28 при  $d = 71$ ). Биты со счётчиком выше порога переворачиваются, синдром обновляется, и так до нулевого синдрома.

### ? Почему декодер может ошибиться и чем это опасно?

Счётчики – случайные величины, и их распределения перекрываются. При неудачном расположении ошибок декодер не сходится. Для пользователя это отказ расшифрования, а для криптостойкости – утечка: по статистике отказов восстанавливается ключ (атака реакцией). Защита – одноразовые ключи или декодер с DFR порядка  $2^{-128}$ .

### ? Зачем нужна маска $a$ , если структура и так скрыта?

Без неё  $c_0 = m + e_0$ : сообщение, в котором перевёрнуто около 67 бит из 12 323, то есть текст читается почти целиком. Маска  $a$  – аналог матрицы  $S$  у Гоппы: кодируется  $ma$ , и открытый текст в шифротексте не виден.

### ? Почему маску можно держать в открытом ключе?

$a^{-1}$  может вычислить любой, но  $c_0 a^{-1} = m + e_0 a^{-1}$ , а  $e_0 a^{-1}$  – плотный шум. Снять его можно, только найдя  $e_0$ , то есть решив задачу декодирования без ключа.

### ? Почему здесь не нужна перестановка $P$ ?

У Гоппы  $P$  скрывает связь позиций с носителем. Здесь секрет – разреженные  $h_0, h_1$ , а по плотному  $q$

их не восстановить: это поиск слова малого веса. Перестановка  $2r$  позиций к тому же разрушила бы квазициклическую структуру, и ключ перестал бы быть компактным.

**? На какой задаче держится стойкость?**

На двух задачах декодирования квазициклических кодов: найти слово веса  $w$  (восстановление ключа) и найти ошибку веса  $t$  по синдрому (расшифрование). Лучшие атаки – ISD; параметры BIKE уровня 1 дают около 128 бит с учётом выигрыша от цикличности.

**? Чем схема в работе отличается от настоящего BIKE?**

BIKE – схема инкапсуляции ключа: шифруется случайная ошибка, общий ключ – хэш от неё, данные шифруются симметрично. Шифротекст –  $r$  бит, ключи одноразовые. В работе – учебная форма Мак-Элиса с шифрованием самого сообщения; она податлива и уязвима к атаке реакцией при многократном использовании ключа.